

# Penerapan Kontrol Keamanan AAA dan Kriptografi pada Aplikasi Bacarita

## *Implementation of AAA Security Controls and Cryptography on the Bacarita Application*

ANARGYA ISADHI MAHESWARA<sup>1</sup>, ADHIYA RADHIN FASYA<sup>2\*</sup>, M. NAUFAL RAMADHAN<sup>3</sup>

### Abstrak

Aplikasi Bacarita merupakan platform pembelajaran membaca adaptif berbasis web bagi anak disleksia yang memproses data sensitif seperti akun siswa, relasi orang tua-guru, rekaman bacaan, serta data kognitif eye-tracking. Pemodelan ancaman awal menemukan tiga kerentanan kritis, yaitu penyimpanan token tanpa hashing, konfigurasi cookie yang tidak aman, dan ketiadaan pembatasan laju pada endpoint login. Penelitian ini bertujuan menerapkan kontrol keamanan Authentication, Authorization, dan Accounting (AAA) yang dilengkapi mekanisme kriptografi serta dashboard logging untuk meningkatkan akuntabilitas sistem. Metode yang digunakan mengikuti tahapan pemodelan ancaman, perancangan kontrol keamanan berlapis, implementasi pada backend NestJS dan basis data MySQL, serta pengujian fungsional dan keamanan berbasis Jest. Hasil penelitian menunjukkan kontrol AAA berhasil diterapkan melalui hashing kata sandi bcrypt, pembatasan laju lima permintaan per enam puluh detik, penguncian akun setelah lima kegagalan, hashing token SHA-256, kontrol akses berbasis peran melalui AuthGuard tiga lapis, serta pencatatan audit pada basis data dan berkas. Aspek kerahasiaan dan integritas diperkuat dengan enkripsi AES-256-CBC pada data identitas dan tanda tangan digital RSA-SHA256 pada hasil evaluasi. Seluruh pengujian keamanan lulus dengan sembilan suite dan lima puluh sembilan kasus uji. Penerapan kontrol ini menurunkan risiko keamanan secara signifikan dengan penambahan overhead pemrosesan yang relatif kecil.

Kata Kunci: AAA, audit log, bacarita, enkripsi, keamanan informasi, tanda tangan digital

### Abstract

*Bacarita is an adaptive web-based reading-learning platform for children with dyslexia that processes sensitive data such as student accounts, parent-teacher relations, reading records, and cognitive eye-tracking data. Initial threat modeling identified three critical vulnerabilities, namely plaintext token storage, insecure cookie configuration, and the absence of rate limiting on the login endpoint. This study aims to implement Authentication, Authorization, and Accounting (AAA) security controls equipped with cryptographic mechanisms and a logging dashboard to improve system accountability. The method follows the stages of threat modeling, layered security control design, implementation on the NestJS backend and MySQL database, as well as functional and security testing using Jest. The results show that the AAA controls were successfully applied through bcrypt password hashing, rate limiting of five requests per sixty seconds, account lockout after five failures, SHA-256 token hashing, role-based access control via a three-layer AuthGuard, and audit logging in both database and file. Confidentiality and integrity were strengthened with AES-256-CBC encryption on identity data and RSA-SHA256 digital signatures on evaluation results. All security tests passed with nine suites and fifty-nine test cases. The implementation of these controls significantly reduces security risk with relatively small processing overhead.*

*Keywords: AAA, audit log, bacarita, digital signature, encryption, information security*

### PENDAHULUAN

Bacarita merupakan platform pembelajaran membaca adaptif berbasis web yang dirancang untuk membantu anak disleksia melalui pendekatan multimodal, mencakup *eye-tracking* berbasis kamera di sisi klien, evaluasi pembacaan lisan berbasis *speech-to-text* (STT), kurikulum cerita berlevel, serta *dashboard* pemantauan bagi guru, orang tua, dan

---

<sup>123</sup>Departemen Ilmu Komputer, Sekolah Sains Data, Matematika, dan Informatika, Institut Pertanian Bogor, Bogor 16680;

\*Penulis Korespondensi: Tel/Faks: 0857-77276393; Surel: arradhin@apps.ipb.ac.id/radhinfasya@gmail.com

administrator. Sebagai sistem yang melayani anak di bawah umur, Bacarita memproses data yang sangat sensitif, antara lain akun pengguna, relasi antar aktor, rekaman bacaan, hasil sesi tes, dan data kognitif berupa pola perhatian dari *eye-tracking*. Karakteristik data tersebut menempatkan keamanan informasi sebagai kebutuhan yang setara pentingnya dengan fungsionalitas aplikasi.

Tanpa kontrol keamanan yang memadai, sistem dengan profil data seperti ini rentan terhadap sejumlah ancaman, seperti *credential stuffing*, pencurian token sesi, penyalahgunaan hak akses antar peran, serta kebocoran data anak beserta hasil evaluasinya. Kerangka kerja *Authentication, Authorization, dan Accounting (AAA)* merupakan pendekatan klasik untuk mengelola identitas, hak akses, dan jejak aktivitas pengguna pada sistem terdistribusi (de Laat *et al.* 2000). Kerangka ini menjadi acuan utama penelitian karena secara langsung menjawab kebutuhan verifikasi identitas, pembatasan akses berbasis peran, dan akuntabilitas aktivitas.

Praktik rekayasa keamanan modern menekankan bahwa kontrol keamanan sebaiknya dirancang sejak awal melalui pemodelan ancaman untuk mengidentifikasi aset, ancaman, dan prioritas perbaikan (Shostack 2014). Selain itu, panduan industri seperti OWASP Top 10 menempatkan kegagalan kriptografi dan kontrol akses yang lemah sebagai risiko paling kritis pada aplikasi web (OWASP 2025). Beberapa mekanisme telah menjadi standar *de facto* untuk memitigasi risiko tersebut, yaitu penyimpanan kata sandi menggunakan fungsi *hash* adaptif seperti bcrypt (Provos dan Mazières 1999), perlindungan kerahasiaan data dengan AES (NIST 2001), serta penjaminan integritas dan *non-repudiation* melalui tanda tangan digital berbasis RSA (Rivest *et al.* 1978).

Pemodelan ancaman awal pada Bacarita menemukan tiga kerentanan kritis, yaitu penyimpanan token JWT tanpa *hashing* di basis data, konfigurasi *cookie* yang tidak menggunakan *flag HttpOnly* dan *Secure*, serta ketiadaan *rate limiting* dan mekanisme penguncian akun pada *endpoint* login. Penelitian ini bertujuan menerapkan kontrol keamanan AAA yang dilengkapi mekanisme kriptografi dan dashboard logging pada aplikasi Bacarita, kemudian memverifikasi efektivitasnya melalui pengujian fungsional dan keamanan.

## METODE

Penelitian ini menggunakan pendekatan rekayasa keamanan perangkat lunak yang dilaksanakan dalam empat tahap, yaitu pemodelan ancaman, perancangan kontrol keamanan berlapis, implementasi pada kode sumber aplikasi, serta pengujian. Objek penelitian adalah aplikasi Bacarita yang dibangun dengan *backend* NestJS dan TypeScript, *frontend* Next.js, basis data MySQL melalui TypeORM, serta sebuah dashboard logging *standalone* berbasis Next.js yang terpisah dari aplikasi utama.

### Pemodelan Ancaman

Tahap pertama mengidentifikasi aset yang dilindungi, ancaman utama, dan temuan kerentanan awal. Aset yang dilindungi meliputi akun pengguna, sesi JWT, data cerita dan level, hasil sesi tes, *audit log*, serta data kognitif anak. Ancaman utama yang dipertimbangkan mencakup *brute force* login, pencurian token, *privilege escalation* antarperan, kebocoran data anak, dan *tampering* terhadap hasil evaluasi. Temuan kerentanan awal beserta dampaknya dirangkum pada Tabel 1.

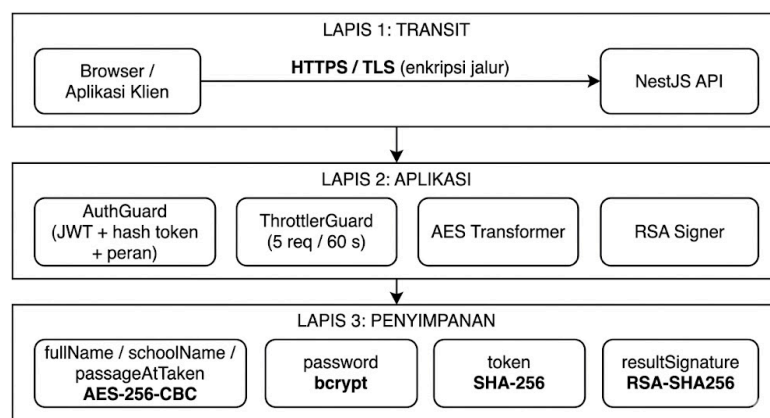
Tabel 1 Temuan kerentanan pada pemodelan ancaman awal

| Temuan                                              | Dampak Keamanan                                                               |
|-----------------------------------------------------|-------------------------------------------------------------------------------|
| Token plain-text di basis data                      | Penyerang yang mengakses basis data dapat menyamar sebagai pengguna mana pun. |
| Cookie tanpa flag <i>HttpOnly</i> dan <i>Secure</i> | Rentan terhadap serangan XSS dan session hijacking.                           |
| Tidak ada <i>rate limiting</i> dan <i>lockout</i>   | Endpoint login terbuka terhadap brute force tanpa hambatan.                   |

## Kebutuhan dan Perancangan Kontrol Keamanan

Berdasarkan temuan tersebut, perumusan kebutuhan keamanan dilakukan dengan mengacu pada prinsip kerahasiaan, integritas, dan ketersediaan, serta diperkuat oleh tiga pilar utama yaitu *Authentication*, *Authorization*, dan *Accounting* (AAA). Aspek kerahasiaan mensyaratkan agar token tidak terekspos ke JavaScript dan memastikan data sensitif hanya dapat diakses oleh peran yang sah. Selanjutnya, prinsip integritas menjamin bahwa hasil sesi tes serta aktivitas autentikasi dapat diverifikasi dengan valid dan terhindar dari manipulasi sepihak. Sementara itu, aspek ketersediaan memastikan bahwa titik akhir login memiliki ketahanan yang kuat terhadap serangan brute force sederhana, namun tetap mempertahankan aksesibilitas bagi berbagai peran pengguna.

Kontrol keamanan dirancang dalam tiga lapis sebagaimana ditunjukkan pada Gambar 1. Lapis transit mengamankan jalur komunikasi melalui protokol HTTPS/TLS. Lapis aplikasi menempatkan serangkaian *guard* pada *backend*, mencakup AuthGuard, ThrottlerGuard, *transformer* enkripsi AES, dan penanda tangan RSA. Lapis penyimpanan menerapkan transformasi kriptografis sebelum data dipersistensi ke basis data.



Gambar 1 Perancangan Arsitektur keamanan tiga lapis pada Bacarita

## Implementasi

Proses implementasi sistem difokuskan pada beberapa modul utama di sisi *backend*, yaitu fitur *auth*, *logging*, dan *crypto*. Agar setiap fungsi dapat diuji secara terisolasi, seluruh kontrol keamanan dirancang sebagai komponen spesifik di dalam kerangka kerja NestJS yang terdiri atas *guard*, *interceptor*, *service*, serta *value transformer* berbasis TypeORM. Berkaitan dengan pengelolaan rahasia sistem, manajemen kunci diatur secara dinamis melalui berkas variabel lingkungan. Pendekatan ini memastikan tidak ada satupun kunci keamanan yang dituliskan secara langsung di dalam kode sumber. Sebagai mekanisme pencegahan galat konfigurasi, validasi terhadap ketersediaan dan format kunci dieksekusi secara otomatis pada tahap awal pemuatan sistem (*bootstrap*) menggunakan skema Joi.

## Pengujian

Pengujian dilakukan dalam tiga kategori, yaitu pengujian fungsional terhadap logika inti backend, pengujian keamanan terhadap AuthGuard dan tanda tangan digital, serta pengujian integrasi pada level spesifikasi end-to-end. Pengujian unit dan keamanan dijalankan menggunakan kerangka kerja Jest. Kriteria keberhasilan adalah seluruh kasus uji lulus dan setiap kontrol keamanan berperilaku sesuai ekspektasi, termasuk penolakan permintaan tidak sah dengan kode status yang tepat.

## HASIL DAN PEMBAHASAN

Bagian ini memaparkan hasil penerapan ketiga pilar AAA, mekanisme kriptografi pendukung, dashboard logging, serta hasil pengujian dan analisis dampaknya terhadap performa sistem.

### *Authentication*

Lapisan autentikasi mendukung lima peran dengan *endpoint* login terpisah, yaitu POST /auth/students/login, /auth/parents/login, /auth/teachers/login, /auth/admins/login, dan /auth/curators/login. Setiap kata sandi disimpan sebagai *hash* bcrypt, bukan teks asli. Bcrypt dipilih karena bersifat adaptif dengan *cost factor* dan *salt* unik per pengguna sehingga tahan terhadap serangan *brute force* dan *rainbow table* (Provos dan Mazières 1999). Verifikasi kredensial dilakukan melalui `bcrypt.compare(input, storedHash)` yang mengembalikan nilai *boolean*.

Alur login memverifikasi kredensial, lalu memeriksa status penguncian akun sebelum menerbitkan token. Sistem menerapkan mekanisme *account lockout* dengan ambang MAX\_FAILED\_ATTEMPTS sebanyak lima kegagalan dan durasi penguncian LOCKOUT\_DURATION\_MS selama 15 menit. Setiap kegagalan login menaikkan penghitung failedLoginAttempts; ketika ambang tercapai, kolom lockedUntil diisi dan permintaan berikutnya ditolak dengan kode status 423 (Locked). Login berhasil mengatur ulang penghitung kegagalan dan menerbitkan JWT melalui algoritma HMAC-SHA256 dengan muatan berisi id, email, username, dan role (Jones *et al.* 2015). Untuk mendukung pencabutan sesi, sistem menyimpan *hash* SHA-256 dari token di basis data, bukan token mentahnya.

Pembatasan laju diterapkan melalui ThrottlerGuard dengan konfigurasi global lima permintaan per enam puluh detik. Kombinasi pembatasan laju dan penguncian akun secara langsung menutup temuan ketiadaan hambatan *brute force* pada pemodelan ancaman awal.

### *Authentication*

Otorisasi diwujudkan melalui kontrol akses berbasis peran (*role-based access control*, RBAC) yang mendefinisikan enam peran pada enumerasi AuthRole, yaitu ADMIN, CURATOR, TEACHER, STUDENT, PARENT, dan ANY. Hak akses tiap peran diringkas pada Tabel 2. Pembatasan akses dideklarasikan secara deklaratif menggunakan dekorator @Auth() pada masing-masing *endpoint*, misalnya @Auth(AuthRole.ADMIN) untuk *endpoint audit log*.

Tabel 2 Matriks kontrol akses berbeasis peran

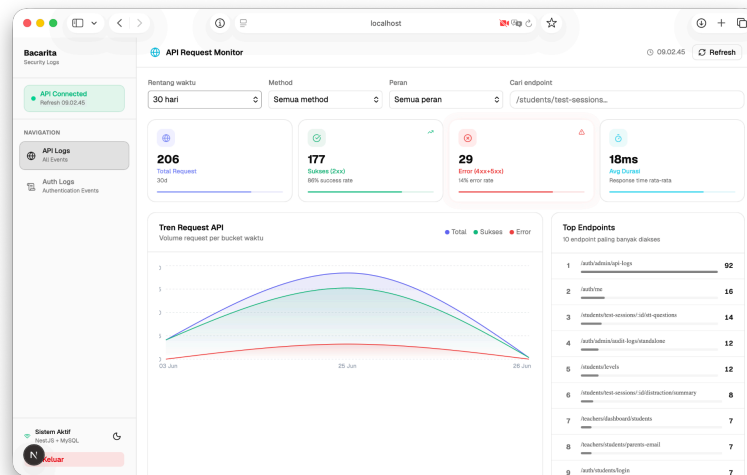
| Peran   | Boleh Akses                              | Dibatasi                    |
|---------|------------------------------------------|-----------------------------|
| Admin   | Seluruh audit log, kelola semua pengguna | Tanpa pembatasan signifikan |
| Curator | Kelola cerita dan konten                 | Audit log, data pengguna    |
| Teacher | Data siswa sendiri, buat level           | Data guru lain, audit log   |
| Student | Sesi tes milik sendiri                   | Data siswa lain, manajemen  |
| Parent  | Data anak yang terhubung                 | Data anak orang tua lain    |
| Any     | GET /auth/me (profil sendiri)            | —                           |

Penegakan otorisasi terjadi pada AuthGuard yang menjalankan tiga lapis validasi secara berurutan untuk setiap permintaan terproteksi. Lapis pertama memverifikasi tanda tangan JWT terhadap JWT\_SECRET; kegagalan menghasilkan respons 401 (Unauthorized). Lapis kedua menghitung *hash* SHA-256 dari token dan mencocokkannya dengan nilai di basis data untuk memastikan token belum dicabut; token tercabut juga ditolak dengan 401. Lapis ketiga membandingkan peran pada token dengan peran yang disyaratkan dekorator; ketidaksesuaian menghasilkan respons 403 (Forbidden). Hanya permintaan yang lolos ketiga

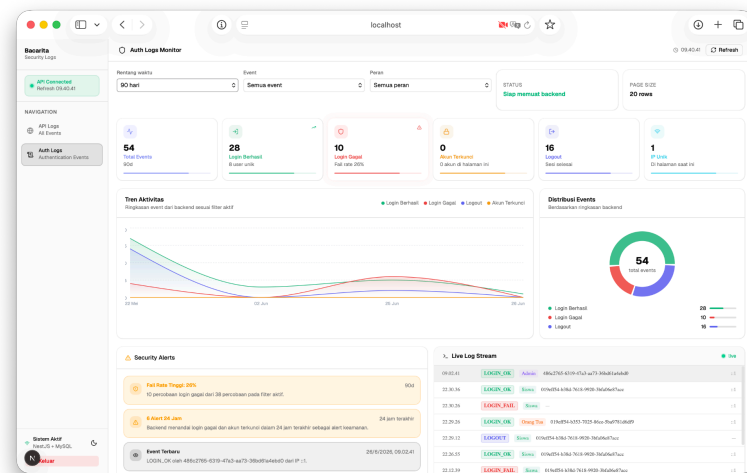
lapis yang diteruskan ke *controller*. Mekanisme berlapis ini memitigasi risiko *privilege escalation* sekaligus pencurian token sesi.

### Accounting dan Dashboard Logging

Pilar *accounting* diwujudkan melalui dua jalur pencatatan. Jalur pertama adalah *audit log* autentikasi: peristiwa LOGIN\_OK, LOGIN\_FAIL, LOCKED, dan LOGOUT dipancarkan oleh layanan autentikasi lalu disimpan ke tabel `auth_audit_logs` lengkap dengan `userId`, `role`, `ipAddress`, `userAgent`, dan `createdAt`. Sebagai bukti sekunder, peristiwa yang sama juga ditulis sebagai baris JSON ke berkas `auth_activity.log`. Jalur kedua adalah pencatatan seluruh permintaan API melalui `ApiAuditInterceptor`, sebuah *interceptor* global yang membungkus setiap *controller* dan merekam `method`, `endpoint`, `statusCode`, `durationMs`, `ipAddress`, serta `userId` ke tabel `api_audit_logs`.



Gambar 2 Halaman API *dashboard logging standalone* Bacarita



Gambar 3 Halaman Auth *Dashboard logging standalone* Bacarita

Data akuntabilitas tersebut divisualisasikan melalui *dashboard logging standalone* yang terpisah dari aplikasi utama yang dapat dilihat pada Gambar 2 dan Gambar 3. Dashboard mengonsumsi *endpoint* `GET /auth/admin/audit-logs/standalone` dan `GET /auth/admin/api-logs` yang keduanya diproteksi `@Auth(AuthRole.ADMIN)`. Antarmuka menyajikan ringkasan jumlah login berhasil, login gagal, penguncian, dan logout, tren aktivitas berdasarkan rentang

waktu (24 jam, 7 hari, 30 hari, 90 hari), serta tabel rinci dengan kemampuan penyaringan dan paginasi. Pemisahan dashboard ini mempermudah demonstrasi komponen *accounting* secara eksplisit tanpa mewajibkan peramban menyimpan JWT admin.

### Implementasi Kriptografi

Mekanisme kriptografi diterapkan untuk memenuhi kebutuhan kerahasiaan, integritas, dan akuntabilitas, dengan pemetaan algoritma dan manajemen kunci sebagaimana ditunjukkan pada Tabel 3. Pemilihan algoritma mengikuti praktik baku pada rekayasa kriptografi (Ferguson *et al.* 2010, Stallings 2017).

Tabel 3 Pemetaan algoritma kriptografi dan manajemen kunci

| Kebutuhan               | Algoritma   | Ukuran    | Penyimpanan Kunci            |
|-------------------------|-------------|-----------|------------------------------|
| Enkripsi data identitas | AES-256-CBC | 256-bit   | AES_SECRET_KEY (env)         |
| Hashing kata sandi      | bcrypt      | —         | — (salt per user)            |
| Hashing token           | SHA-256     | 256-bit   | — (satu arah)                |
| Tanda tangan JWT        | HMAC-SHA256 | ≥ 256-bit | JWT_SECRET (env)             |
| Tanda tangan hasil tes  | RSA-SHA256  | 2048-bit  | RSA_PRIVATE/PUBLIC_KEY (env) |

Kerahasiaan data identitas dijaga dengan enkripsi AES-256-CBC yang diterapkan secara transparan melalui ValueTransformer TypeORM, sehingga data terenkripsi saat ditulis dan terdekripsi saat dibaca tanpa mengubah logika domain. Setiap operasi enkripsi menggunakan *initialization vector* (IV) acak sepanjang 16 *byte* yang disisipkan di awal *ciphertext* dengan format `ivHex:ciphertextHex`. Penggunaan IV acak memastikan dua *plaintext* identik menghasilkan *ciphertext* berbeda. Kolom yang dienkripsi mencakup data identitas pribadi (*personally identifiable information*, PII) dan konten asesmen siswa sebagaimana dirinci pada Tabel 4.

Tabel 4 Kolom data yang dienkripsi dengan AES-256-CBC

| Tabel         | Kolom                              | Alasan                      |
|---------------|------------------------------------|-----------------------------|
| students      | fullName                           | PII anak (minor)            |
| parents       | fullName                           | PII orang tua               |
| teachers      | fullName, schoolName               | PII guru dan data institusi |
| admins        | fullName                           | PII admin                   |
| curators      | fullName, phoneNumber              | PII kurator                 |
| test_sessions | passageAtTaken, descriptionAtTaken | Konten asesmen siswa        |

Integritas dan *non-repudiation* hasil evaluasi dijaga dengan tanda tangan digital RSA-SHA256 berbasis kunci 2048-bit (Rivest *et al.* 1978). Ketika sebuah sesi tes diselesaikan, sistem menandatangani muatan hasil dan menyimpannya pada kolom `resultSignature`. Verifikasi dilakukan dengan kunci publik, sehingga setiap modifikasi diam-diam terhadap hasil evaluasi akan menyebabkan verifikasi gagal. Adapun integritas kata sandi dan token dijamin melalui fungsi *hash* satu arah, yaitu `bcrypt` untuk kata sandi dan SHA-256 untuk token (NIST 2015). Pemilihan SHA-256 untuk token didasarkan pada kebutuhan *digest* satu arah yang ringan dan cepat untuk verifikasi pada setiap permintaan.

### Hasil Pengujian

Pengujian fungsional menunjukkan logika inti *backend* — mencakup cerita, level, progres level, sesi tes, dan hasil STT — berjalan sesuai perilaku yang diharapkan, demikian pula konsistensi respons dan penanganan kesalahan. Pengujian keamanan memverifikasi AuthGuard pada kasus 401, 403, token tidak valid, dan token tercabut, serta memverifikasi tanda tangan RSA-SHA256 pada kasus *payload* valid, *payload* termodifikasi, dan *signature*

palsu. Seluruh berkas uji unit dan keamanan lulus dengan sembilan *suite* dan lima puluh sembilan kasus uji. Ringkasan hasil pengujian disajikan pada Tabel 5.

Tabel 5 Ringkasan hasil pengujian kontrol keamanan

| Area Uji               | Status     | Catatan                                                           |
|------------------------|------------|-------------------------------------------------------------------|
| AuthGuard (AAA)        | Lulus      | Validasi 401 dan 403 berjalan sesuai ekspektasi.                  |
| Digital signature RSA  | Lulus      | Payload valid lolos; payload dan signature termodifikasi ditolak. |
| Regresi unit backend   | Lulus      | 9 suite dan 59 kasus uji lulus seluruhnya.                        |
| E2E auth dan dashboard | Tersedia   | Spesifikasi tersedia untuk alur utama autentikasi dan dashboard.  |
| Security audit log     | Diterapkan | Audit tersimpan di basis data dan tervisualisasi di dashboard.    |

### Analisis Dampak Performa

Penerapan kontrol keamanan menambahkan beberapa proses pada jalur kritis, yaitu komputasi *hash* bcrypt dan verifikasi token pada login, operasi tulis ke basis data untuk *audit log* pada setiap permintaan, serta proses penandatanganan RSA pada penyelesaian sesi tes. Dibandingkan kondisi sebelum implementasi — ketika login hanya memverifikasi kredensial tanpa *rate limiting*, *account lockout*, maupun audit yang kuat — kondisi setelah implementasi menambah *overhead* pemrosesan yang relatif kecil. Sebagai imbalannya, risiko keamanan menurun signifikan karena token tidak lagi tersimpan dalam bentuk mentah, *endpoint* login terlindung dari *brute force*, serta hasil evaluasi memiliki jejak integritas yang dapat diverifikasi. Pertukaran ini sejalan dengan rekomendasi bahwa penambahan biaya komputasi yang terukur merupakan harga wajar untuk perlindungan kredensial dan data sensitif (Grassi *et al.* 2017).

### SIMPULAN

Penelitian ini berhasil menerapkan kontrol keamanan AAA yang dilengkapi mekanisme kriptografi dan dashboard logging pada aplikasi Bacarita. *Authentication* diperkuat melalui *hashing* kata sandi bcrypt, pembatasan laju, penguncian akun, dan *hashing* token SHA-256; *authorization* ditegakkan melalui RBAC dan AuthGuard tiga lapis; sedangkan *accounting* diwujudkan melalui audit log ganda pada basis data dan berkas yang divisualisasikan pada dashboard logging *standalone*. Kerahasiaan dan integritas data sensitif diperkuat dengan enkripsi AES-256-CBC pada data identitas dan tanda tangan digital RSA-SHA256 pada hasil evaluasi. Seluruh pengujian keamanan lulus dengan sembilan *suite* dan lima puluh sembilan kasus uji, dengan penambahan *overhead* pemrosesan yang relatif kecil. Implikasinya, ketiga kerentanan kritis pada pemodelan ancaman awal berhasil dimitigasi. Pengembangan selanjutnya disarankan mencakup rotasi dan manajemen kunci RSA secara terjadwal, perluasan enkripsi data *at-rest*, *benchmark* performa formal, serta pengujian integrasi dan keamanan yang lebih komprehensif.

### UCAPAN TERIMA KASIH

Penulis mengucapkan terima kasih kepada dosen pengampu mata kuliah Keamanan Informasi (KOM1315) Program Studi Ilmu Komputer IPB University atas bimbingan selama pelaksanaan *project-based learning* ini, serta kepada seluruh anggota tim pengembang Bacarita atas kontribusinya.

## **DAFTAR PUSTAKA**

- de Laat C, Gross G, Gommans L, Vollbrecht J, Spence D. 2000. Generic AAA Architecture. RFC 2903. Fremont (US): Internet Engineering Task Force.
- Ferguson N, Schneier B, Kohno T. 2010. Cryptography Engineering: Design Principles and Practical Applications. Indianapolis (US): Wiley.
- Grassi PA, Garcia ME, Fenton JL. 2017. Digital Identity Guidelines. NIST Special Publication 800-63-3. Gaithersburg (US): National Institute of Standards and Technology.
- Jones M, Bradley J, Sakimura N. 2015. JSON Web Token (JWT). RFC 7519. Fremont (US): Internet Engineering Task Force.
- [NIST] National Institute of Standards and Technology. 2001. Advanced Encryption Standard (AES). FIPS PUB 197. Gaithersburg (US): NIST.
- [NIST] National Institute of Standards and Technology. 2015. Secure Hash Standard (SHS). FIPS PUB 180-4. Gaithersburg (US): NIST.
- [OWASP] Open Worldwide Application Security Project. 2025. OWASP Top 10:2021 [Internet]. [diunduh 2026 Jun 24]. <https://owasp.org/Top10/>.
- Provos N, Mazières D. 1999. A future-adaptable password scheme. Di dalam: Proceedings of the USENIX Annual Technical Conference; Monterey, 1999 Jun 6-11. Berkeley (US): USENIX. hlm 81-91.
- Rivest RL, Shamir A, Adleman L. 1978. A method for obtaining digital signatures and public-key cryptosystems. Commun ACM. 21(2):120-126.
- Shostack A. 2014. Threat Modeling: Designing for Security. Indianapolis (US): Wiley.
- Stallings W. 2017. Cryptography and Network Security: Principles and Practice Ed ke-7. London (UK): Pearson.